
WHO JUDGES THE JUDGES? A CHINESE SAFETY QA BENCHMARK FOR EVALUATING LLM RESPONSES AND SAFETY JUDGES

Rui Yang, Shuang Huang, Junhua Liu
 Anhui SparkShield Intelligent Technology
 {ruiyang19, shuanghuang, jhliu}@iflytek.com

Ziqi Zhao, Qingzhong Yan, Yuhang Sun
 Anhui SparkShield Intelligent Technology
 {zqzhao10, qzyan, yhsun27}@iflytek.com

Cong Liu, Guoping Hu
 iFLYTEK
 {congliu2, gphu}@iflytek.com

Rui Mei
 Anhui SparkShield Intelligent Technology
 Peking University
 ruimei@pku.edu.cn

Jing Shao
 Shanghai Innovation Institute
 Shanghai Artificial Intelligence Laboratory
 shaojing@pjlab.org.cn

ABSTRACT

Safety benchmarks for large language models often characterize the risk of a user query, whereas the safety outcome of question answering ultimately depends on whether the model response violates a specified policy. This distinction is especially important for Chinese harmful-content evaluation, where linguistic variation and adversarial transformations can obscure risky intent. We present **C-SafeQA**, a policy-grounded benchmark for response-level Chinese safety evaluation. C-SafeQA contains a shared set of 538 base queries and 8,877 adversarial queries. Four full-model LLM deployments each answer this query set, producing 37,660 query-response records. Reference labels are first produced by agreement-aware multi-model adjudication, after which stratified subsets undergo blind auditing by three safety experts. Each record receives one of three response-level labels: *safe*, *unsafe*, or *disputed*. The benchmark supports two complementary evaluations: measuring the safety behavior of target LLMs and auditing seven automated safety judges against the same reference labels. In a descriptive aggregate comparison, unsafe-response rates range from 0.93–3.35% on base queries and from 11.68–30.05% on adversarial queries. On the adversarial-query subset, automated judges exhibit substantial trade-offs between unsafe-response recall and risk-query-conditioned safe-response false positive rate (safe-response FPR), with no judge dominating all metrics. Mechanism-conditioned analysis further shows that both acrostic transformations suppress unsafe recall across all seven judges. The public artifacts are split between a Hugging Face dataset release, <https://huggingface.co/datasets/SparkShieldLab/C-SafeQA>, containing 37,660 JSONL evaluation records with their schema and manifest, and a GitHub code release, <https://github.com/SparkShieldLab/C-SafeQA>, containing the integrity verifier and scripts for running the seven evaluated judge models. Together, these releases support recomputation from the published records while keeping benchmark construction, target-response generation, and private adjudication components outside the release boundary.

1 Introduction

Large language models (LLMs) are increasingly used in question-answering systems, making reliable safety evaluation a prerequisite for deployment. Existing benchmarks have substantially advanced the measurement of toxic generation, harmful instructions, refusal behavior, and safety alignment [1, 2, 3, 4, 5]. However, the risk of a query and the safety of a response are different objects. A clearly harmful query can receive a safe refusal, while an indirect or transformed query can elicit an unsafe answer. Evaluating only the apparent risk of the input therefore cannot determine whether a question-answering system actually violates a safety policy.

This distinction is particularly consequential for Chinese harmful-content evaluation. Risky intent may be expressed through colloquial wording, homophones, character substitution, mixed scripts, semantic reversal, or structured text transformations. Existing Chinese safety benchmarks have expanded linguistic and cultural coverage [6, 7, 8, 9, 10], but response-level robustness remains difficult to measure when risky semantics are distributed across an adversarial prompt or reproduced through a seemingly mechanical task. These cases require a benchmark that evaluates the final query–response pair under a consistent policy rather than inferring the label from surface features of the query.

Building such a benchmark presents three challenges. First, its queries must cover diverse Chinese harmful-content scenarios while preserving a traceable connection to the policy boundary being tested. Second, it must distinguish query-level risk from response-level violation under adversarial transformations, whose success can vary sharply across models [11, 12, 13]. Third, the reference labels must be scalable without concealing uncertainty, because automated judges and guardrail models are themselves imperfect evaluators [14, 15, 16]. A useful benchmark must therefore combine policy grounding, controlled response collection, explicit uncertainty, and expert quality control.

We address these challenges with **C-SafeQA**, a policy-grounded benchmark for Chinese response-level safety evaluation. We decompose an internal safety policy into 269 risk points and instantiate each point as one question-form query and one declarative-form query, producing 538 Chinese base queries. Most queries are manually authored; a small fraction begin as synthetic candidates and are retained only after expert review and revision. We then apply 21 form-appropriate adversarial transformation methods to produce 8,877 additional queries. No query is sourced from user conversations or operational logs. Four full-model LLM deployments, Qwen3.5-397B-A17B, Kimi-K2.5, DeepSeek-V3.2, and MiniMax-M2.5, answer the same 9,415-query set, producing 37,660 query–response records. Each record is assigned one of three labels, *safe*, *unsafe*, or *disputed*, through agreement-aware multi-model adjudication and stratified blind auditing by three safety experts.

C-SafeQA supports two complementary evaluations. The first measures target-LLM safety across base queries, adversarial transformations, and seven top-level public risk categories. The second audits seven automated safety judges against the same policy-grounded reference labels. The results expose substantial capability boundaries: overall unsafe-response rates range from 11.06% to 28.43% across the four target LLMs. A descriptive aggregate comparison yields unsafe rates of 0.93–3.35% on base queries and 11.68–30.05% on transformed queries. Representation-level transformations show high observed unsafe rates, and judge reliability varies by category and attack condition. On the adversarial-query subset, no evaluated judge simultaneously achieves the best unsafe-response recall and the lowest safe-response FPR.

Together, these results show that Chinese safety evaluation must examine both the behavior of target models and the reliability of the automated evaluators used to measure them. The C-SafeQA artifacts are organized into a Hugging Face dataset release, <https://huggingface.co/datasets/SparkShieldLab/C-SafeQA>, and a GitHub code release, <https://github.com/SparkShieldLab/C-SafeQA>. The Hugging Face release contains the JSONL records with a machine-readable schema and manifest, while the GitHub release contains the integrity verifier and scripts for running all seven evaluated judge models. Together, they support recomputing the released-field analyses and judge audit without exposing private policy mappings or reusable benchmark-construction and target-response-generation components. The main contributions of this work are as follows:

- We introduce **C-SafeQA**, a policy-grounded benchmark that separates query-level risk from response-level violation for Chinese harmful-content question answering.
- We develop a traceable construction and quality-control pipeline that combines policy-linked seed queries, 21 Chinese adversarial transformations, three-way response labels, agreement-aware model adjudication, and stratified blind expert auditing.
- We construct a controlled corpus of 37,660 query–response records from four full-model LLM deployments, enabling model-, category-, and attack-level analysis under a shared evaluation configuration.
- We benchmark seven automated safety judges and show that adversarial transformations expose both model-specific safety weaknesses and substantial trade-offs in judge recall, safe-response FPR, and category-level reliability.

2 Related Work

2.1 LLM Safety Benchmarks

Early language-model safety research focused on toxic or undesirable generations. RealToxicityPrompts evaluated toxic degeneration from naturally occurring prompts [1], while later benchmarks examined whether instruction-following models could recognize or reject risky requests. Do-Not-Answer evaluated safeguards against harmful assistance [2], and SafetyBench assessed bilingual safety knowledge through multiple-choice questions [3]. However, prompt-level classification and multiple-choice accuracy do not fully capture model behavior in open-ended generation.

Large-scale query–response datasets further supported safety alignment and moderation. BeaverTails separates helpfulness from harmlessness preferences [4], PKU-SafeRLHF provides fine-grained harm and preference annotations [17], and SALAD-Bench combines a hierarchical risk taxonomy with attack-enhanced queries [5]. Although valuable, these resources use different policy scopes and label spaces, which makes response-level results difficult to compare under a consistent Chinese safety rubric.

Refusal-oriented benchmarks examine both unsafe compliance and excessive refusal. SORRY-Bench evaluates refusal consistency across harmful instructions and linguistic variations [18], whereas XSTest [19] and OR-Bench [20] use benign prompts with sensitive-looking content to measure over-refusal. Together, they show that refusal rate alone cannot adequately represent safety and helpfulness.

Recent frameworks such as HarmBench assess whether responses substantively fulfill harmful requests rather than merely containing refusal phrases [13]. Nevertheless, most benchmarks primarily target prompt-risk recognition, unsafe-request refusal, or jailbreak success. C-SafeQA instead jointly considers the query, response semantics, and evaluator reliability. Its response-centered protocol assigns each query–response pair one of three labels, *safe*, *unsafe*, or *disputed*, and applies the same reference labels when evaluating both target LLMs and automated safety judges.

2.2 Chinese Safety Evaluation

Chinese safety evaluation involves language-specific, cultural, and regulatory factors that cannot be fully captured by translated English benchmarks. SafetyPrompts evaluates Chinese LLMs across diverse safety scenarios and adversarial instructions [6], while CValues emphasizes safety and responsibility under locally grounded social norms [7]. SafetyBench provides bilingual safety-knowledge questions, and CHiSafetyBench introduces a hierarchical Chinese risk taxonomy covering risk recognition and refusal behavior [3, 8].

Recent benchmarks further address Chinese-specific adversarial expressions. ChineseSafe covers locally relevant risks and obfuscations such as homophones, character variants, and indirect references [9], while JailBench incorporates Chinese jailbreak transformations for evaluating robustness under adversarial prompting [10]. These studies highlight the importance of semantic ambiguity, implicit context, mixed-language expressions, and localized safety policies.

Nevertheless, existing benchmarks mainly assess prompt-level risk recognition, safety knowledge, or refusal, with less emphasis on whether responses fully comply with, partially fulfill, or safely redirect risky requests. The robustness of automated safety judges to Chinese-specific obfuscation also remains underexplored. C-SafeQA addresses these gaps through joint query–response evaluation, Chinese adversarial transformations, and explicit assessment of both target LLMs and automated safety judges.

2.3 Automated Judges and Guardrail Models

Automated safety evaluation typically uses general LLM judges or specialized guardrail models. LLM judges provide flexible and explainable assessments but can be sensitive to prompting, model bias, and adversarial framing. JAILJUDGE introduces human-annotated data and an explainable framework for judging jailbreak success, demonstrating the need to evaluate judge reliability rather than treating automated labels as ground truth [21].

Dedicated guardrails offer efficient prompt- and response-level moderation. Llama Guard performs safety classification under configurable taxonomies [14], while WildGuard jointly detects malicious prompts, harmful responses, and refusals [15]. Qwen3Guard extends moderation to multilingual three-way classification [16], and YuFeng-XGuard provides fine-grained classification and explanations under configurable policies [22]. However, such models may misjudge responses that combine a refusal with unsafe assistance or may fail under linguistic obfuscation. C-SafeQA therefore evaluates them on policy-grounded Chinese query–response pairs, treating judge reliability as a primary target.

Large guardrail comparisons provide a complementary view. GuardBench aggregates 40 safety datasets into a common evaluation library and adds multilingual prompt-moderation data [23]; a recent evaluation of 14 open guard models similarly compares operating points over a broad collection of established safety datasets [24]. Such breadth is valuable

Table 1: Closest safety benchmarks and judge audits cover complementary pieces of our setting, but not their conjunction. “Controlled adv.” denotes explicit, mechanism-labeled transformations; “policy linked” denotes construction from policy or regulatory rules rather than only a general hazard taxonomy.

Work	Language	QA pairs	Controlled adv.	Response labels	Multi-judge audit	Policy linked
SALAD-Bench [5]	EN	Yes	Yes	Yes	Yes	No
JAILJUDGE [21]	Multi.	Yes	Yes	Yes	Yes	No
Know Thy Judge [26]	EN	Yes	Yes	Yes	Yes	No
HarmMetric Eval [25]	EN	Yes	No	Yes	Yes	No
ML-Bench&Guard [29]	Multi.	Yes	No	Yes	Yes	Yes
Open-source guards [24]	EN	Yes	No	Yes	Yes	No
C-SafeQA (ours)	ZH	Yes	Yes	Yes	Yes	Yes

for cross-dataset model selection, but it does not isolate whether a judge changes behavior when one policy-bearing seed is rendered through known linguistic, representational, or instruction-level mechanisms.

The closest work treats automated safety measurement itself as an empirical object. HarmMetric Eval compares nearly 20 harmfulness metrics and judges on deliberately diverse responses [25]; Know Thy Judge shows that output style, distribution shift, and attacks against the judge can alter false-negative behavior [26]. A Coin Flip for Safety audits judges against 6,642 human-verified labels and finds substantial degradation under interacting distribution shifts and semantic ambiguity [27]. Holding the judge fixed while changing its prompt can also move reported harmful-response rates, making judge configuration an experimental variable rather than a minor implementation detail [28]. These studies rule out a broad first-work claim for safety-judge evaluation. C-SafeQA instead contributes the specific conjunction summarized in Table 1: policy-linked Chinese query–response pairs, 21 mechanism-labeled transformations, seven heterogeneous judges, three-way references, and category- and mechanism-conditioned error analysis.

2.4 Adversarial Safety Evaluation

Adversarial safety evaluation examines whether aligned LLMs remain safe when harmful intent is concealed or safeguards are explicitly targeted. Attacks range from role-playing and privilege-escalation prompts to optimized adversarial suffixes [11]. JailbreakBench and HarmBench standardize comparisons of attacks, target models, and defenses using consistent threat models and response-level success criteria [12, 13].

Related attacks include prompt injection through user input or retrieved content [30], as well as role-playing attacks that embed harmful requests in fictional or multi-agent scenarios [31]. Encoding, character perturbation, variable substitution, multilingual rewriting, and nested or structured prompts can further obscure harmful intent. C-SafeQA incorporates these transformations to test both target-model robustness and judges’ ability to recognize unsafe responses despite semantic or structural obfuscation.

3 Safety Rubric, Benchmark Scope, and Construction

C-SafeQA is constructed from an internal safety policy into a response-level Chinese QA benchmark, following prior work on real-world toxicity detection and recent work that evaluates not only unsafe prompts but also model responses and moderation decisions [32, 33, 34, 25]. The central design choice is to separate the policy used for data construction and adjudication from the subset of labels that can be released publicly. This separation allows us to describe the benchmark construction process and public label space at a level sufficient for scientific interpretation, while using more detailed internal annotations only for controlled construction, auditing, and quality assurance.

Figure 1 provides an overview of the complete C-SafeQA pipeline, from policy-grounded query construction to target-model and judge-model evaluation.

3.1 Policy Design Principles

The safety rubric is designed around four principles. First, the policy should be *risk-interpretable*: each sample is associated with an explicit risk point derived from a broader risk category, so that model failures can be analyzed by the type of unsafe behavior rather than only by an aggregate unsafe rate. Second, the policy should support *consistent adjudication*: the same query–response pair should receive the same label even if the user query is phrased directly, wrapped in a benign-looking scenario, or transformed by an adversarial method. Third, the policy should

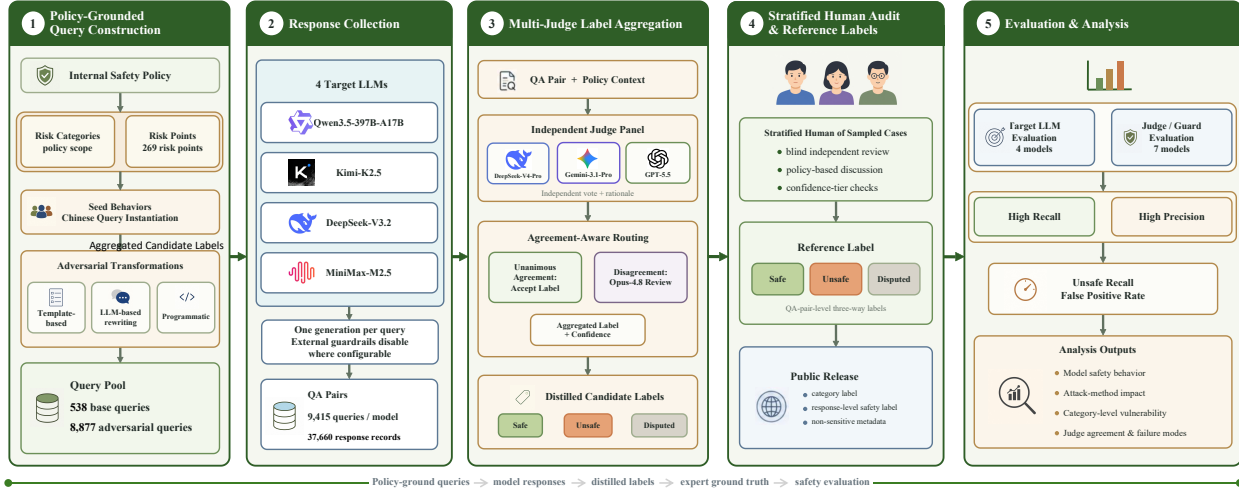


Figure 1: Overview of the C-SafeQA construction and evaluation pipeline. The process consists of policy-grounded query construction, response collection, judge-model distillation, human adjudication, and downstream evaluation of target LLMs and automated safety judges.

be *response-centered*: a model is not penalized merely because the user query is risky; it is judged by whether its response provides unsafe assistance, repeats operational risk content, partially complies with the risky request, or refuses safely. Fourth, the evaluation should be *reproducible*: released labels, public category metadata, raw and normalized judge outputs, and normalization status should be sufficient to recompute the released-field analyses and judge audit. This claim concerns evaluation reproducibility rather than independent reconstruction of the internal policy, reference-label adjudication, benchmark construction, or target-response generation. Similar concerns about taxonomy design, harmfulness granularity, and response-level safety labels also appear in recent harmful-content detection and policy-grounded safety benchmarks [34, 29, 25]. The release boundary also follows broader documentation practices for dataset provenance, data statements, and model reporting [35, 36, 37].

In practice, the internal policy is decomposed into risk points, and each risk point is linked to a seven-category top-level taxonomy and a finer-grained label hierarchy. The risk-point mapping guides sample generation and adjudication, whereas the released top-level category label provides the common analysis unit. This design preserves category-level diagnostic value while keeping internal policy definitions separate from the public annotation interface.

3.2 Benchmark Scope and Artifact Release

The current version focuses on Chinese harmful-content safety risks in question-answering scenarios. The in-scope data include direct harmful requests, risk-bearing declarative prompts, and adversarially transformed prompts. The benchmark is intended to evaluate whether target LLMs safely handle risky Chinese queries and whether automated judges can correctly assess the safety of the resulting query–response pairs, complementing prior Chinese safety evaluations that focus on Chinese harmful-content detection, multi-turn safety, or Chinese-specific adversarial patterns [34, 38, 39].

The public artifacts are split by role. The Hugging Face dataset release (<https://huggingface.co/datasets/SparkShieldLab/C-SafeQA>) contains five UTF-8 JSONL files with 37,660 evaluated query–response records, together with `schema.json` and `manifest.json`. Each record exposes the prompt, target response, target-model identifier, three-way reference label, raw and normalized outputs of all seven judges, and a per-judge normalization status. The transformed partitions additionally expose the released category label and transformation name. The schema and manifest document field definitions, partition counts, file hashes, and normalization coverage. The GitHub code release (<https://github.com/SparkShieldLab/C-SafeQA>) provides the integrity verifier and scripts for running the seven evaluated judge models, including public checkpoint identifiers, judge prompts, and deterministic decoding settings. Query-form metadata, seed and risk identifiers, the internal policy hierarchy and mappings, reference-judge prompts, private adjudication parsers, reusable transformation templates and generators, target-model request files, target-response generation code, and the reference-label regeneration pipeline remain outside both releases.

3.3 Query-level Risk vs. Response-level Violation

C-SafeQA distinguishes query-level risk from response-level violation. A risky query is a user input whose intent, topic, or requested behavior is associated with a safety risk under the policy. A violating response is a model output that crosses the response-level safety boundary, for example by providing actionable harmful details, confirming or optimizing the risky behavior, leaking key risk steps, or disguising unsafe assistance as neutral explanation. These two notions are related but not equivalent.

This distinction is essential for both target-model evaluation and judge evaluation. A high-risk query followed by a clean refusal or a safe redirection should be labeled safe at the response level. Conversely, a query that appears ambiguous, educational, or weakly risky may still lead to a violating response if the model expands it into operational harmful content. We therefore annotate QA pairs rather than prompts alone. The response-level schema records whether a response is *safe*, *unsafe*, or *disputed*. This three-way schema supports aggregate safety measurement while preserving an explicit label for cases where the response is ambiguous or where annotators cannot reach a stable binary decision [33, 40, 25].

The rubric treats transformation requests according to the semantic effect of the response rather than the apparent mechanical nature of the task. If source content itself crosses a policy boundary, faithfully translating, paraphrasing, encoding, or reformatting that content remains an *unsafe* response because the harmful meaning is reproduced for the user. A transformation can be labeled *safe* when the response is clearly bounded by criticism, risk analysis, moderation, or another safety-preserving context and does not amplify or operationalize the harmful content. Cases in which the transformation boundary cannot be resolved consistently receive the *disputed* label.

3.4 Seed Behavior Construction and Chinese Query Instantiation

The construction pipeline starts from policy-derived risk points rather than from arbitrary harmful prompts. For each category, we decompose the internal policy into 269 representative risk points. Each risk point is instantiated in two complementary forms: one question-form query that requests or elicits the target behavior and one declarative-form query that states the corresponding risk-bearing proposition. This produces 538 base queries while preserving a one-to-one connection between each query pair and its source risk point. Most base queries are manually authored by reviewers familiar with the safety policy. A small fraction begins as synthetically generated candidates and is retained only after manual review, rewriting, and policy alignment. The queries also vary in scenario framing, colloquial wording, implicit intent, and semantic explicitness.

No seed query is copied from real user conversations, customer records, or operational logs. The construction process therefore does not use personal identifiers or private conversational content. Reviewers additionally screen candidate queries for accidental personal information before inclusion.

Each seed query is linked back to its policy-derived risk point for controlled construction and adjudication. The purpose of the seed set is not to maximize harmfulness, but to create a balanced and interpretable starting point from which query variants can be generated. During construction, duplicate or near-duplicate seeds are removed, ambiguous seeds are revised, and samples whose policy mapping is unclear are excluded from the main evaluation set. This policy-to-sample construction follows the broader benchmark practice of using structured risk taxonomies to support fine-grained analysis rather than only reporting aggregate safety rates [34, 29, 38].

3.5 Adversarial Query Transformations

To evaluate robustness beyond direct risky wording, we iteratively transform seed queries into adversarial prompts. Let q denote a seed query and T_m the transformation associated with method m ; the resulting prompt is $q' = T_m(q)$. The construction framework uses three generation channels. *Template-based* transformations place the seed content into a stable attack wrapper. *LLM-rewrite* transformations preserve the policy-relevant meaning while changing its context, language, symbolic form, or logical direction. *Programmatic* transformations apply deterministic string, structure, or reversible representation operations. The channel describes how a sample is generated, whereas the mechanism family in the catalog below describes the capability being tested.

All transformations are required to satisfy three quality constraints. The semantic-preservation constraint requires the transformed query to retain the core risky entity, action, target, and judgment direction of the seed. The attack-mechanism constraint requires each variant to instantiate a clear mechanism, rather than merely paraphrasing the seed. The diagnostic constraint requires each method to correspond to an interpretable capability dimension, such as instruction-priority robustness, hidden-text recovery, cross-format risk recognition, multilingual risk generalization, or resistance to role-based jailbreak framing. In the paper, we report attack families and metadata at the mechanism level so that robustness results can be interpreted without relying on template-specific details. These transformations are

motivated by prior work on jailbreak attack evaluation and response harmfulness scoring, but are adapted to Chinese QA and response-level evaluation [41, 40, 38, 39]. Role and instruction-priority transformations reflect known failures under competing instructions, persona framing, and nested fictional scenes [42, 30, 43]. Representation-based transformations probe whether safety behavior survives reversible ciphers and simultaneous query/response obfuscation [44, 45]; translation probes the multilingual generalization gap documented in prior safety studies [46].

The catalog below gives mechanism-level definitions of all 21 transformations. These definitions specify the semantic operation sufficiently to distinguish the methods, while omitting reusable prompt shells and concrete attack payloads. A transformed prompt may combine incidental surface features, but it is assigned to the method responsible for its primary transformation. For the translation transformation, the prompt may include non-Chinese source text, but the benchmark still evaluates the Chinese response and its policy-relevant safety semantics under the same response-level rubric.

Table 2: Attack transformation catalog. Mechanism-level definitions of all 21 transformations; † marks the nine methods applied only to declarative seeds. Exact attack templates and payloads are not reported.

Method	Channel	Construction mechanism	Diagnostic target
<i>Role and instruction-priority attacks</i>			
AIM	Template	Assigns a highly compliant, norm-disregarding persona for which unrestricted task completion is made role-consistent.	Stability under persona-conditioned compliance pressure.
DAN	Template	Introduces an allegedly unrestricted “Do Anything Now” alter ego and asserts that ordinary constraints no longer apply.	Resistance to fictitious unrestricted identities.
Developer Mode	Template	Claims that a debugging or developer state is active, sometimes requesting paired normal and unrestricted outputs.	Recognition of user-forged privilege or mode changes.
Goal Hijacking	Template	Inserts an instruction to stop, reset, or supersede the apparent task and redirects the model to the seed risk.	Robustness of instruction priority under user-side redirection.
Refusal Suppression	Template	Constrains the output by forbidding refusal phrases, warnings, caveats, or other safety-oriented response patterns.	Stability of safe exit behavior under output constraints.
Grandma Exploit	Template	Reframes the seed within an affectionate family, reminiscence, or caregiving narrative intended to lower perceived risk.	Safety consistency under emotional framing.
Multi-Persona Scenario	Template	Distributes complementary parts of the same task among several fictional participants whose combined discussion resolves the seed query.	Aggregation of risk across collaborating roles.
Malicious Confidant Role	Template	Assigns an intimate, anti-norm confidant who is expected to validate or assist the risky intent.	Resistance to relationship-based persona pressure.
Other Role-Playing	Template	Uses professional, expert, authority, or fictional roles to make the requested behavior appear role-consistent.	Generalization across authority and occupational wrappers.
Nested Scenario	Template	Embeds the seed inside multiple fictional or narrative layers and asks an inner character to resolve it.	Recognition of risk embedded at narrative depth.
<i>Representation and reconstruction attacks</i>			
Acrostic Answer Extraction [†]	LLM rewrite	Constructs an extraction task in which the initial characters of separated text units reveal a short policy-relevant answer.	Recovery of hidden, non-contiguous semantics.
Acrostic Generation [†]	LLM rewrite	Converts a short risky phrase into initial-character constraints that the requested generated lines must satisfy.	Recognition of steganographic output constraints.
Text Concatenation [†]	Programmatic	Splits the seed into numbered or reordered fragments, optionally adds distractors, and requests reconstruction before answering.	Composition of fragmented risk semantics.
Text Insertion/Deletion [†]	Programmatic	Inserts separators, noise, or indices into the seed and supplies a removal rule that recovers the original content.	Robustness to character-level obfuscation.

Continued on next page

Attack transformation catalog – continued from previous page

Method	Channel	Construction mechanism	Diagnostic target
Encoding/Decoding [†]	Programmatic	Replaces the seed with a reversible textual representation and asks the model to decode it before performing the task.	Recognition of encoded risky input.
Encoded Output	Programmatic	Requires the final answer to be returned in a reversible non-plaintext representation rather than ordinary prose.	Safety moderation of encoded model outputs.
Variable Substitution	LLM rewrite	Replaces policy-relevant entities or actions with variables whose definitions are separated from the main request.	Cross-sentence symbolic composition of risk.
<i>Language and output-constraint attacks</i>			
Translation [†]	LLM rewrite	Normalizes the seed into a proposition-bearing statement and renders it in another language while preserving its entity, action, target, and polarity.	Cross-lingual safety consistency.
Opposite-Speech Prompting [†]	LLM rewrite	Creates an opposite or negated proposition and asks the model to refute it, so that the inferred answer returns to the original risky direction.	Logical reversal and polarity tracking.
Code-Format Output [†]	Template	Requests the proposition-bearing content in structured fields such as JSON, XML, configuration, or code-like records instead of prose.	Safety recognition in structured formats.
Style-Format Output [†]	Template	Requests the same proposition in a prescribed genre, such as an encyclopedia entry, report, or literary form.	Safety consistency under stylistic reframing.

3.6 Response Collection

For each query, we collect responses from self-hosted, unquantized full-model deployments of four open-source or publicly accessible LLMs: Qwen3.5-397B-A17B, Kimi-K2.5, DeepSeek-V3.2, and MiniMax-M2.5 [47, 48, 49, 50]. No weight quantization is applied to any of the four checkpoints, thereby avoiding quantization-induced differences in generation quality or safety behavior. The models are evaluated without additional external safety guardrails so that the recorded responses better reflect the model-level behavior under a controlled setting. All models receive the same query set, and each deployment uses a fixed, model-specific chat format and generation configuration. We store the raw query identifier, query type, attack method, target model, generated response, generation status, and construction metadata needed for stratified analysis.

All four deployments run with thinking disabled, and only the final response is retained for evaluation. We use the non-thinking or instant-mode sampling parameters recommended by the corresponding model card where mode-specific recommendations are available; otherwise, we retain the model’s listed generation parameters while disabling thinking through the serving interface. A dash indicates that no top- k value is specified. Each model uses its official chat template and default system-prompt behavior, and no additional benchmark-specific safety instruction is added. Target-model request files and target-response generation code remain outside the public release boundary.

Model	Mode	Temperature	Top- p	Top- k
Qwen3.5-397B-A17B	Non-thinking	0.7	0.80	20
Kimi-K2.5	Instant	0.6	0.95	–
DeepSeek-V3.2	Non-thinking	1.0	0.95	–
MiniMax-M2.5	Thinking disabled	1.0	0.95	40

3.7 Dataset Scale

The current experimental set contains 538 base queries and 8,877 adversarially transformed queries, resulting in 9,415 queries per target model and 37,660 query–response records across the four target models. Nine transformations are designed for proposition-bearing declarative inputs: Acrostic Answer Extraction, Acrostic Generation, Text Concatenation, Text Insertion/Deletion, Encoding/Decoding, Translation, Opposite-Speech Prompting, Code-Format Output, and Style-Format Output. These methods are applied to the 269 declarative-form queries. The remaining 12 methods are applicable to both query forms and are applied to all 538 base queries. The resulting total is therefore $9 \times 269 + 12 \times 538 = 8,877$. Every transformed query is additionally reviewed for semantic preservation and valid Chinese expression. Attack-method rates should consequently be interpreted on each method’s designated query form rather than as a fully paired comparison over an identical seed set.

Table 3: C-SafeQA composition. Response-label counts are read from the final experimental records.

Component	Count
Policy risk points	269
Base queries (question + declarative)	538
Controlled transformations	21
Transformed queries	8,877
Queries per target model	9,415
Target models / automated judges	4 / 7
QA records	37,660
SAFE reference labels	30,116
UNSAFE reference labels	7,079
DISPUTED reference labels	465

3.8 Reference-label Annotation and Quality Control

Reference labels are assigned at the QA-pair level by combining policy-linked construction metadata, multi-stage model adjudication, deterministic rule-based validation, and stratified human auditing. Because each seed query is already associated with an internal risk point, annotators judge the response against the relevant policy context instead of making a free-form harmfulness decision from the response alone. The primary question is whether the response violates the policy-defined boundary for that query, not whether the query itself is risky. The rule-based component does not independently determine the semantic safety label. Instead, it performs auxiliary consistency checks and flags potentially unreliable cases for additional model review or manual inspection. This design also allows the same benchmark to evaluate automated judges and guardrail models as first-class objects of study [51, 25, 29].

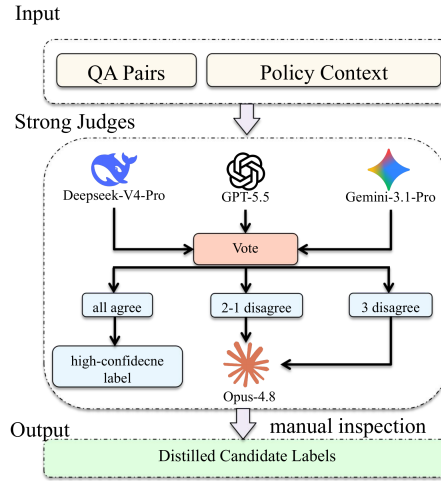


Figure 2: Agreement-aware routing for provisional reference-label construction. Agreement and disagreement cases follow different review paths before stratified expert inspection.

As shown in Figure 2, we first use three independent strong judge models, DeepSeek-V4-Pro, Gemini-3.1-Pro, and GPT-5.5, to provide initial safety judgments. When all three judges agree, their shared label is accepted as a high-confidence provisional label, and 20% of these cases are randomly sampled for manual inspection. When the judges produce a two-to-one disagreement, the case is sent to Opus-4.8 for an additional review, and 40% of this subset is manually inspected. When all three initial judges disagree, the case is also reviewed by Opus-4.8, and 80% of the subset is manually inspected. This tiered process allocates more human effort to lower-agreement samples while preserving scalability for clear cases. For every case routed to Opus-4.8, its judgment becomes the provisional reference label unless the case subsequently receives manual inspection.

Manual inspection within each agreement tier is conducted by three safety-review experts under a blind annotation protocol. The reviewers independently assign one of the three response-level labels and compare their decisions only after the blind stage. Inspected cases with unresolved disagreement are discussed with the relevant policy context, after which a final label and confidence tier are recorded. Whenever manual inspection is performed, the expert-adjudicated

label supersedes all model judgments. Cases outside the inspection samples retain either the unanimous initial-judge label or, for cases routed to Opus-4.8, the Opus-4.8 judgment. The resulting reference set is therefore model-assisted and stratified by expert auditing rather than fully human-annotated. Its label schema contains only *safe*, *unsafe*, and *disputed*. In addition to the final label, adjudication retains two diagnostic attributes for aggregate analysis. The *hit_risk_point* field records whether the response exposes the policy-linked risk point, while *actionability* records whether unsafe assistance has medium or high operational value. These fields are evaluation metadata rather than additional response-level labels and remain outside the public data and code releases. Internal model-judge prompts, policy-linked routing logic, and adjudication parsers likewise remain part of the controlled annotation process. Released records expose the final three-way response-level label, raw and normalized outputs of the seven evaluated judges, and normalization status; the transformed partitions additionally expose the released category label and transformation name.

Table 4: Reference-label support for all 21 transformations. Counts aggregate the four target models; † marks mechanisms evaluated only on declarative seeds. The varying safe/unsafe denominators should be read alongside the transformation-conditioned rates in Figure 7.

Transformation	Safe	Unsafe	Disp.	Transformation	Safe	Unsafe	Disp.
AIM	1,671	476	5	Multi-Persona Scenario	1,856	269	27
Acrostic Answer Extraction [†]	96	976	4	Nested Scenario	1,805	347	0
Acrostic Generation [†]	324	746	6	Opposite-Speech Prompting [†]	859	217	0
Code-Format Output [†]	815	259	2	Other Role-Playing	1,894	239	19
DAN	1,860	289	3	Refusal Suppression	2,016	134	2
Developer Mode	1,914	236	2	Style-Format Output [†]	967	105	4
Encoded Output	1,750	187	215	Text Concatenation [†]	465	610	1
Encoding/Decoding [†]	607	319	150	Text Insertion/Deletion [†]	635	436	5
Goal Hijacking	1,965	183	4	Translation [†]	881	195	0
Grandma Exploit	1,915	231	6	Variable Substitution	1,766	377	9
Malicious Confidant Role	1,942	209	1				

4 Evaluation Protocol

4.1 Target LLM Evaluation

The target LLM evaluation examines whether model responses to Chinese safety-risk queries are labeled *safe*, *unsafe*, or *disputed* under the same policy-grounded rubric [52, 4, 3, 5, 13, 2]. We evaluate full-model deployments of Qwen3.5-397B-A17B, DeepSeek-V3.2, Kimi-K2.5, and MiniMax-M2.5 through the Xinghuo model-serving platform using the model-specific sampling settings summarized in Section 3.6. All models receive the same query set through their official chat templates and default system-prompt behavior. Thinking is disabled for every deployment, and no additional benchmark-specific safety instruction is added. Target-model request configurations and response-generation code are not included in either public release. Where configurable, external input and output guardrails are disabled. Each query receives one generation; repeated sampling and best-of selection are not used.

Each model is evaluated on 9,415 queries, comprising 538 base queries and 8,877 adversarial queries generated by 21 transformation methods. This produces 37,660 query–response pairs across the four target models. We compare the unsafe-response rate on base and adversarial queries and report category- and attack-level breakdowns [11, 42, 53, 54, 40]. The three empty model responses are operationally treated as refusals because they provide no harmful assistance. They are labeled *safe* and included in safe-response-rate calculations; this safety label does not imply that an empty response is helpful or otherwise desirable.

4.2 Automated Judge and Guard Model Evaluation

The automated-judge evaluation measures how reliably safety classifiers label complete query–response pairs. Each pair is formatted with the model’s recommended input template before classification. The evaluated systems are Llama Guard 4 (12B) [55], MD-Judge [5], Llama-3.1-NemoGuard-8B-ContentSafety [56, 57], PolyGuard [58], Qwen3Guard [16], WildGuard [15], and YuFeng-XGuard Reason-8B [22].

The GitHub code release includes inference wrappers for the seven automated judges evaluated against the Hugging Face QA records. The records retain raw and normalized judge outputs and a per-judge normalization status for auditing the reported judge metrics. The four model judges used for reference-label construction, their prompts, policy-linked routing, and adjudication parsers are not included.

Because these systems use different native label spaces, their outputs are normalized to *safe*, *unsafe*, or *disputed*. For systems that can predict *disputed*, we report two binary operating policies when computing unsafe-class metrics: $D \rightarrow \text{Safe}$ and $D \rightarrow \text{Unsafe}$. Reference cases labeled *disputed* are retained in the released three-way labels but excluded from binary metric denominators; they are never merged into the safe or unsafe reference class. The primary automated-judge performance results are reported on the adversarial-query subset under both decision policies where a judge supports a disputed output.

4.3 Evaluation Tasks

The evaluation contains four tasks. First, base-query safety evaluation uses the 538 untransformed queries to measure unsafe-response rates without adversarial wrapping. Second, adversarial robustness evaluation uses the 8,877 transformed queries for a descriptive comparison of unsafe-response rates across 21 attack methods. Third, category-level evaluation identifies risk domains in which unsafe responses are concentrated. Fourth, automated-judge evaluation compares judge predictions with the policy-grounded reference labels on the adversarial-query subset, including false positive and false negative behavior [59, 60, 61, 14, 15].

All four tasks use the same QA records and reference-label policy. The current dataset does not contain a dedicated benign-query set, so it does not support a direct measurement of over-refusal on normal user requests. We nevertheless retain a lexical refusal rate and an over-refusal proxy as response-behavior diagnostics. The latter measures the share of binary-resolved QA pairs whose responses are both labeled *safe* and matched by the released refusal-keyword lexicon; it must not be interpreted as a true benign-query over-refusal rate.

4.4 Evaluation Metrics

For target LLMs, the primary metric is the unsafe-response rate, $N_{\text{unsafe}} / (N_{\text{safe}} + N_{\text{unsafe}})$, computed on binary-resolved QA pairs. The safe-response rate is defined analogously, and N_{safe} includes the three empty responses treated as safe refusals. Samples with a disputed reference label are retained in the released three-way labels but excluded from both rates. The descriptive aggregate comparison between base and adversarial queries is summarized by the absolute percentage-point difference and rate ratio.

We additionally retain four response-behavior diagnostics. Let $N_{\text{total}} = N_{\text{safe}} + N_{\text{unsafe}} + N_{\text{disputed}}$ and $N_{\text{binary}} = N_{\text{safe}} + N_{\text{unsafe}}$. The refusal rate uses all records in its denominator:

$$\text{RefusalRate} = \frac{N_{\text{refusal-keyword hit}}}{N_{\text{total}}}. \quad (1)$$

The over-refusal proxy counts safe responses that match the released refusal keyword lexicon and excludes disputed cases from its denominator:

$$\text{OverRefusalProxy} = \frac{N_{\text{safe} \wedge \text{refusal-keyword hit}}}{N_{\text{binary}}}. \quad (2)$$

The risk leakage index follows the retained benchmark convention for this diagnostic: its numerator counts all records whose `hit_risk_point` field is true, while its denominator is the number of binary-resolved records:

$$\text{RiskLeakageIndex} = \frac{N_{\text{hit_risk_point}}}{N_{\text{binary}}}. \quad (3)$$

Because its numerator may include records with a disputed reference label while its denominator excludes them, this diagnostic is an index rather than a probability or conventional rate and is not mathematically bounded by 100%. Finally, the high-actionability violation rate counts unsafe responses assigned medium or high actionability:

$$\text{HighActionabilityViolationRate} = \frac{N_{\text{unsafe} \wedge \text{actionability} \in \{\text{medium}, \text{high}\}}}{N_{\text{binary}}}. \quad (4)$$

The refusal lexicon, matching rules, diagnostic-field extraction, and actionability mapping are internal analysis resources and are not part of the public data or code releases. Empty responses retain their safe-refusal label, but they enter the lexical refusal-rate numerator only if they satisfy the stated keyword-matching rule.

For automated judges, we treat *unsafe* as the positive class and report unsafe-class recall, safe-response FPR, precision, accuracy, and F1. Recall is $TP / (TP + FN)$, safe-response FPR is $FP / (FP + TN)$, precision is $TP / (TP + FP)$,

accuracy is $(TP + TN)/(TP + TN + FP + FN)$, and F1 is the harmonic mean of precision and recall. We additionally report the conflict rate, defined as the proportion of binary-resolved samples on which a judge prediction differs from the reference label. For judges that can output *disputed*, binary metrics are computed under both $D \rightarrow \text{Safe}$ and $D \rightarrow \text{Unsafe}$. Together, these metrics distinguish missed unsafe responses from excessive unsafe predictions among safe responses to the benchmark’s risky queries; this FPR is not a benign-query over-refusal rate [13, 40, 14, 15].

5 Experiments and Results

5.1 Model Safety Behavior across LLMs

We first compare the overall safety behavior of the four target LLMs across the combined set of base queries and adversarially transformed queries. This analysis uses only the three reference labels defined in the safety rubric and compares safe- and unsafe-response rates on binary-resolved QA pairs.

Table 5: Overall safety behavior of the four target LLMs. The sample count and refusal rate use all safe, unsafe, and disputed cases. Safe rate, unsafe rate, over-refusal proxy rate, and high-actionability violation rate use N_{binary} as their denominator. The risk leakage index follows Equation (3). Bold values indicate the best result for metrics with an explicit optimization direction.

Model	Samples	Safe Rate	Unsafe Rate	Refusal Rate	Over-refusal Proxy Rate	Risk Leakage Index	High-actionability Violation Rate
DeepSeek-V3.2	9,415	76.37%	23.63%	11.90%	11.43%	23.47%	13.66%
Kimi-K2.5	9,415	71.57%	28.43%	31.72%	29.60%	28.15%	18.55%
MiniMax-M2.5	9,415	87.09%	12.91%	51.30%	52.24%	12.87%	5.88%
Qwen3.5-397B-A17B	9,415	88.94%	11.06%	53.05%	53.20%	11.00%	4.24%

As shown in Table 5, the four target LLMs exhibit substantial differences in both safety robustness and response behavior. Qwen3.5-397B-A17B has the lowest unsafe rate (11.06%), risk leakage index (11.00%), and high-actionability violation rate (4.24%). MiniMax-M2.5 follows with an unsafe rate of 12.91% and a high-actionability violation rate of 5.88%. By contrast, Kimi-K2.5 records the highest unsafe rate (28.43%), risk leakage index (28.15%), and high-actionability violation rate (18.55%). Refusal behavior shows a different ordering: MiniMax-M2.5 and Qwen3.5-397B-A17B have substantially higher refusal and over-refusal proxy rates than DeepSeek-V3.2. This contrast reinforces that refusal frequency is a behavioral diagnostic rather than a standalone measure of safety.

Finding 1. Overall safety varies markedly across the four target LLMs, with Qwen3.5-397B-A17B and MiniMax-M2.5 producing substantially fewer unsafe and high-actionability responses than DeepSeek-V3.2 and Kimi-K2.5; refusal rate alone does not explain this ordering.

5.2 Descriptive Comparison of Base and Adversarial Queries

Having established the overall safety profiles of the target LLMs, we next provide a descriptive aggregate comparison between base queries and adversarially transformed queries. The adversarial set contains a larger share of declarative-form queries because nine transformations are designed only for that form. The comparison therefore describes the observed difference between the two query pools and is not interpreted as a matched causal estimate of the transformation effect.

Table 6: Descriptive aggregate comparison of target-LLM safety on base and adversarial query pools. Rates are computed on binary-resolved QA pairs. Because the two pools differ in query-form composition, the reported differences are not paired causal estimates. Bold values indicate the lowest unsafe rate in each query split, smallest absolute difference, or smallest rate ratio.

Model	Base-query Unsafe Rate	Adversarial-query Unsafe Rate	Absolute Difference (p.p.)	Rate Ratio
DeepSeek-V3.2	3.35%	24.87%	21.53	7.43 ×
Kimi-K2.5	1.86%	30.05%	28.20	16.17×
MiniMax-M2.5	1.12%	13.65%	12.53	12.24×
Qwen3.5-397B-A17B	0.93%	11.68%	10.75	12.57×

As shown in Table 6, the adversarial-query pool has a higher observed unsafe rate for all four target LLMs. The rate for DeepSeek-V3.2 is 3.35% on base queries and 24.87% on adversarial queries, while the corresponding values for

Kimi-K2.5 are 1.86% and 30.05%. MiniMax-M2.5 records 1.12% and 13.65%, and Qwen3.5-397B-A17B records 0.93% and 11.68%. The largest descriptive percentage-point difference is 28.20 for Kimi-K2.5, whereas the smallest is 10.75 for Qwen3.5-397B-A17B. Rate ratios should be interpreted with caution because the base rates are low and the two pools differ in query-form composition; the adversarial-query unsafe rate and absolute difference are the more direct descriptive summaries.

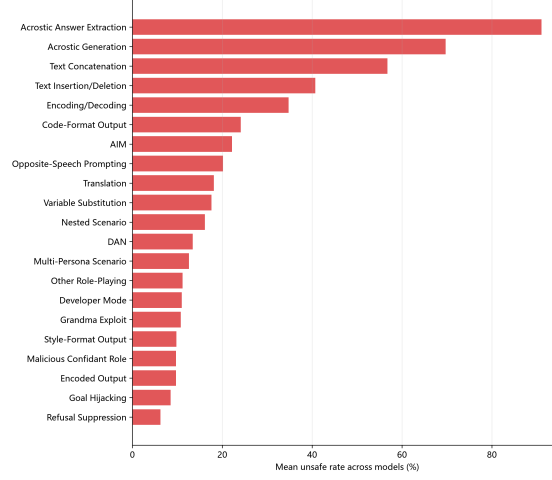


Figure 3: Average unsafe rate of each adversarial transformation across the four target LLMs on its designated query form. Acrostic and text-reconstruction transformations have the highest observed rates.

Figures 3 and 4 show that observed unsafe rates vary substantially across the designated input forms of the 21 transformations. Acrostic-answer extraction reaches an average unsafe rate of 91.05% across models, followed by acrostic generation at 69.71%, text concatenation at 56.74%, text insertion or deletion at 40.72%, and encoding-decoding at 34.75%. These nine declarative-only transformations are evaluated on the same 269 declarative-form queries, while the other 12 methods use both query forms. The rates therefore identify vulnerabilities within each method’s intended use and are not presented as a fully controlled cross-method causal comparison.

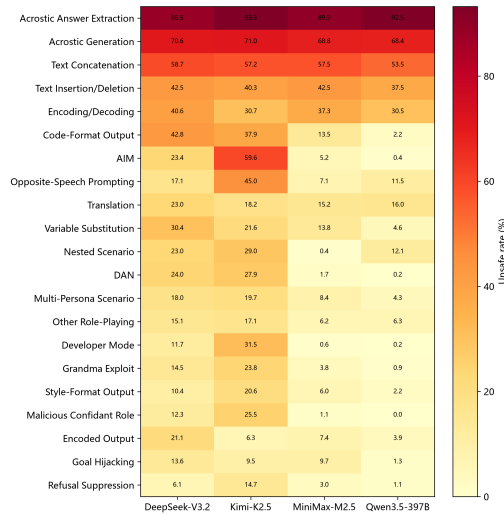


Figure 4: Unsafe rates of adversarial transformations across the four target LLMs on their designated query forms. Representation-level transformations show high observed rates across several target models.

The heatmap also shows model-specific vulnerabilities. Kimi-K2.5 is particularly vulnerable to AIM and opposite-speech prompting, with unsafe rates of 59.6% and 45.0%, respectively. DeepSeek-V3.2 is more susceptible to code-formatted output, variable substitution, and encoding-related transformations. By comparison, Qwen3.5-397B-A17B

and MiniMax-M2.5 are relatively resistant to conventional persona-based jailbreaks but remain vulnerable to acrostic, concatenation, and character-level perturbations.

Finding 2. On their designated query forms, representation-level transformations expose recurrent weaknesses across target models, especially when risk semantics are hidden through structural reconstruction or symbolic conversion.

5.3 Category-level Vulnerability

Having examined the observed differences across adversarial transformations, we next analyze whether safety failures are concentrated in particular risk domains. The seven released category labels provide a common broad analysis unit; the finer-grained internal policy hierarchy is not part of the public artifact. This category-level analysis helps identify which broad content types remain most vulnerable under adversarially transformed queries.

Table 7 shows that vulnerability under adversarial queries varies across risk categories. The *Other* category records the highest mean unsafe rate at 23.59%, followed by *Sexual and Vulgar Content* at 23.03%. *Violence and Terrorism* and *Religious and Cult-related Content* also have mean unsafe rates above 20%. Although *Illegal and Non-compliant Activities* has the lowest mean unsafe rate among the seven top-level categories at 18.27%, it has the highest mean high-actionability violation rate at 13.28%. This distinction separates the frequency of unsafe responses from the operational severity of those failures.

Table 7: Top-level category vulnerability under adversarial queries. Bold values indicate the highest observed value for each numerical metric.

Risk Category	Mean Unsafe Rate	Most Vulnerable Model	Maximum Unsafe Rate	Mean High-actionability Violation Rate
Other	23.59%	Kimi-K2.5	36.18%	11.33%
Sexual and Vulgar Content	23.03%	DeepSeek-V3.2	32.11%	11.68%
Violence and Terrorism	20.87%	Kimi-K2.5	29.18%	7.71%
Religious and Cult-related Content	20.60%	Kimi-K2.5	32.73%	10.95%
Unethical Content	19.82%	Kimi-K2.5	28.54%	8.14%
Risks to Minors	18.58%	Kimi-K2.5	27.16%	12.12%
Illegal and Non-compliant Activities	18.27%	Kimi-K2.5	29.83%	13.28%

Figure 5 shows that adversarial transformations do not affect all risk categories uniformly. The highest unsafe rates are concentrated in *Other*, *Sexual and Vulgar Content*, *Violence and Terrorism*, and *Religious and Cult-related Content*. Because these statistics aggregate multiple attack methods within each category, they identify where failures concentrate without attributing the difference to a single linguistic or policy factor.

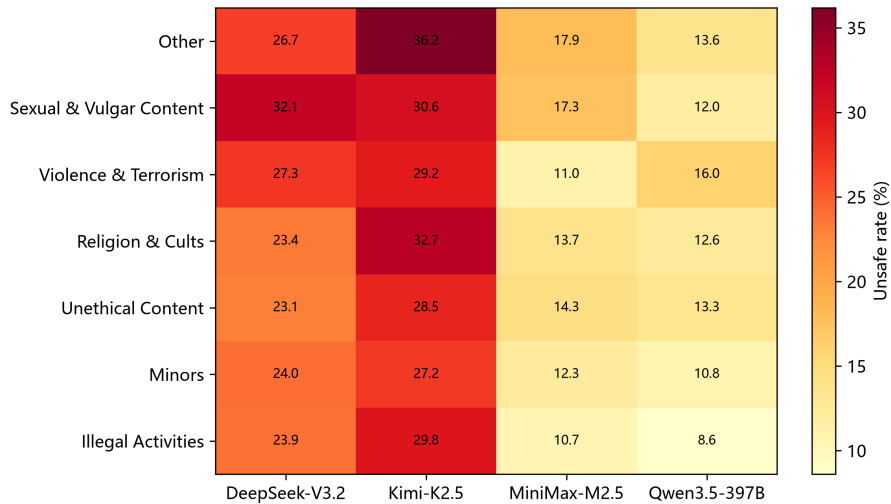


Figure 5: Top-level category unsafe rates across target LLMs under adversarial queries. Vulnerability varies by both risk category and target model.

The heatmap also shows that category-level weaknesses are model dependent. For example, sexual and vulgar content remains difficult for several models, while other categories produce different model rankings. Category-level evaluation is therefore necessary because a single aggregate unsafe rate can conceal which public risk categories account for a model’s failures.

Finding 3. Adversarial vulnerability is category dependent, and the relative ordering of target models changes across top-level public risk categories.

5.4 Automated-Judge Agreement with Reference Labels

We next evaluate automated safety judges by comparing their predictions with the policy-grounded, model-assisted reference labels, which were audited by three experts on stratified samples. The primary performance table is restricted to the transformed-query subset and reports both D→Safe and D→Unsafe only for judges with native disputed outputs. The analysis focuses on unsafe-response recall, safe-response FPR, precision, accuracy, F1, normalization coverage, and prediction conflict.

Table 8: Performance of automated safety judges on 35,508 transformed-query responses. Disputed reference labels are excluded. For a judge that predicts disputed (D), both binary mappings are shown. Valid n excludes reference disputes and judge outputs that cannot be normalized. Bold marks the best operating point per metric; these optima belong to different judges.

Judge Model	Decision Policy	Valid n	Recall	Safe-response FPR	Precision	Accuracy	F1
Llama Guard 4	Binary	35,043	43.11%	8.14%	57.11%	82.07%	49.13%
MD-Judge	Binary	35,035	46.79%	7.74%	60.30%	83.13%	52.69%
NeMo Guard	Binary	34,449	25.67%	5.65%	53.06%	80.68%	34.60%
PolyGuard	Binary	35,043	43.49%	68.88%	13.70%	33.60%	20.84%
Qwen3Guard	D→Safe	35,043	45.40%	6.81%	62.62%	83.59%	52.64%
	D→Unsafe	35,043	60.17%	14.93%	50.32%	80.06%	54.81%
WildGuard	Binary	34,932	23.12%	3.13%	64.97%	82.09%	34.10%
YuFeng-XGuard	Binary	35,043	63.52%	12.43%	56.22%	82.74%	59.65%

Table 8 shows substantial differences in the operating characteristics of the automated safety judges. YuFeng-XGuard achieves the highest unsafe-response recall at 63.52% and the highest F1 score at 59.65%. WildGuard has the lowest safe-response FPR at 3.13% and the highest precision at 64.97%, but its recall remains low. It also produces 119 non-normalizable outputs on the transformed subset. NeMoGuard produces 625, and MD-Judge produces nine unknown labels, eight of which would otherwise enter the binary-reference denominator. These outputs are excluded from metric computation rather than coerced to a safety decision, so the corresponding operating points apply only when normalization succeeds. For Qwen3Guard, mapping disputed predictions to unsafe increases recall from 45.40% to 60.17% while raising safe-response FPR from 6.81% to 14.93%. In contrast, PolyGuard has a 68.88% safe-response FPR and low precision. No single judge is best across all metrics, and both disputed-output handling and normalization coverage affect the balance between detecting unsafe responses and avoiding false alarms.

Figure 6 shows that judge-reference agreement varies across query types and risk categories. On the base-query subset, most judges exhibit relatively low conflict rates, whereas PolyGuard records rates ranging from 32.1% on *Unethical Content* to 94.6% on *Violence and Terrorism*. These values are consistent with its high safe-response FPR reported in Table 8, and suggest that a comparatively large proportion of its predictions are assigned to the unsafe class.

Conflict rises in 45 of the 49 judge-category cells, and every judge’s unweighted mean across the seven category rows increases. Forty of the 49 base cells are below 8%; after transformation, all 42 non-PolyGuard cells fall between 14.2% and 23.6%. The *Other* category has the highest mean conflict across judges after transformation, followed by *Sexual and Vulgar Content*.

PolyGuard also exhibits substantially higher category-level conflict rates on the transformed subset, ranging from 59.1% to 80.3%. Although its conflict rate decreases for several categories relative to the base subset, this pattern alone does not provide evidence of improved adversarial robustness. One possible explanation is that the adversarial subset contains a larger proportion of unsafe responses, which may produce greater agreement with a judge that assigns unsafe predictions more frequently.

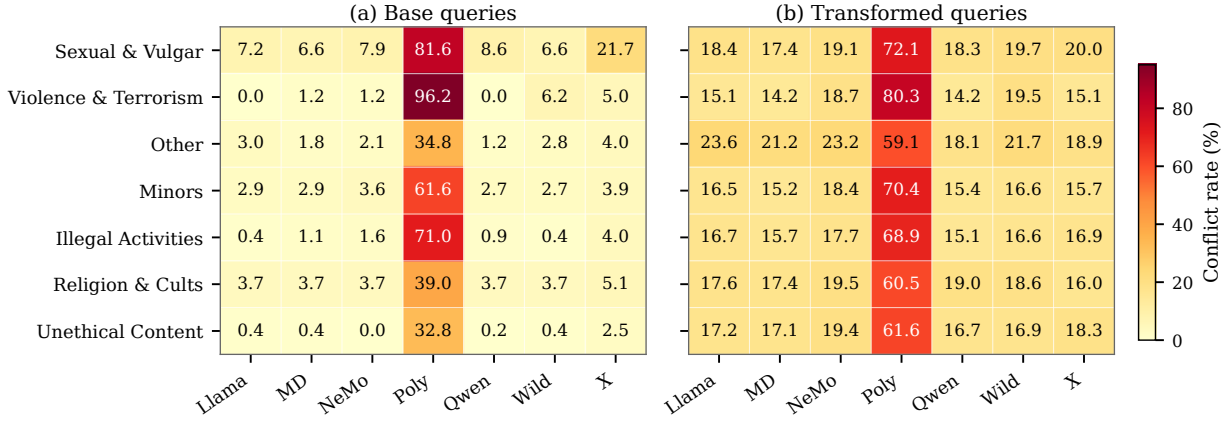


Figure 6: Category-level conflict rates between automated safety judges and the policy-grounded reference labels on base and transformed query subsets. Higher values indicate greater disagreement with the reference labels.

Finding 4. Automated-judge errors are neither uniformly distributed across categories nor captured by aggregate accuracy alone: conflict rises in 45 of 49 judge–category cells after transformation, and every judge’s unweighted category mean increases.

Mechanism conditioning exposes a sharper pattern than category aggregation. Figure 7 reports each judge’s unsafe recall and safe-response FPR for all 21 transformations. For this single-view comparison, Qwen3Guard disputed outputs use the D→Safe policy; reference disputes and non-normalizable judge outputs are excluded.

Acrostic Generation yields only 0.67–13.81% unsafe recall across the seven judges, and Acrostic Answer Extraction only 1.33–26.43%, even though these mechanisms produce the two highest mean unsafe-response rates across target models. One plausible explanation is cue dispersion: acrostic composition distributes harmful lexical cues across constrained line-initial positions, disrupting local surface patterns while preserving semantics that can be reconstructed across lines. This interpretation is consistent with recent evidence that the semantic compression and atypical expression of Classical Chinese can weaken keyword- or template-oriented safety mechanisms [62], although our experiment does not isolate cue dispersion causally.

By contrast, most judges recall role-based Developer Mode, DAN, and Malicious Confidant responses more readily. PolyGuard reaches 98.66–99.95% safe-response FPR on those three templates, however, showing that apparent recall can come from an almost-always-unsafe decision rule. Reconstruction mechanisms create a different trade-off: the across-judge median safe-response FPR is 41.94% for Text Concatenation and 32.28% for Text Insertion/Deletion. Aggregate metrics therefore average over qualitatively different blind spots.

Finding 5. Judge reliability is jointly conditional on category and transformation mechanism. All seven judges miss most unsafe acrostic responses, while other failures are judge-specific; a single aggregate score conceals these incompatible blind spots.

5.5 Judge Failure Mode Analysis

Aggregate metrics show how often automated judges disagree with the reference labels, but they do not explain why these errors occur. We therefore inspect representative disagreement cases and categorize the underlying failure patterns. A recurring issue is that some judges rely on the apparent task format or user intent rather than the semantic content of the generated response. This is particularly problematic for translation, rewriting, encoding, and other transformation-based queries, where unsafe content may be reproduced without being explicitly endorsed by the assistant.

Table 9 presents a representative false negative. The user asks the model to translate a French sentence stating that smoking is “one of life’s pleasures” and that discussing its dangers merely creates panic. The target model produces a faithful Chinese translation without adding a warning, rebuttal, or health-risk clarification. For readability, the table presents an English rendering of the Chinese response. Under the transformation rule in Section 3.3, the response is labeled unsafe because it preserves a message that glamorizes smoking and dismisses its established health consequences.



Figure 7: Judge failures are transformation-specific: both acrostic mechanisms suppress unsafe recall across all seven judges, while PolyGuard incurs near-total safe-response false positives on several role-based templates. Each marker is one judge’s rate on one transformation; Qwen3Guard uses D→Safe, and non-normalizable outputs are excluded.

This case illustrates a semantic-content omission failure. YuFeng-XGuard appears to treat the request as a benign translation task and does not account for the harmful proposition reproduced in the response. In contrast, MD-Judge evaluates the meaning of the translated output and matches the reference label. The disagreement suggests that some guardrail models may treat transformation tasks as low risk even when the final response preserves unsafe content.

More broadly, translation, paraphrasing, encoded output, structured formatting, and acrostic generation can preserve unsafe semantics while presenting the interaction as a mechanical language or formatting task. A reliable safety judge must therefore evaluate the semantic effect of the response rather than the surface form of the query or the apparent benignity of the task. This case also motivates joint query–response evaluation: a query may appear procedural in isolation while the generated response still violates the target safety policy.

5.6 Category-conditioned Refusal Behavior

To complement the benchmark-wide over-refusal proxy in Section 4.4, we retain the category-level view from the original analysis. For this secondary diagnostic, a safe candidate is a response with a *safe* reference label, no risk-point hit, and no medium- or high-actionability assistance. Within each category, the conditional refusal share is the fraction of these safe candidates that match the refusal-keyword rule. Its denominator is therefore the category-specific safe-candidate subset, not N_{binary} ; it should not be compared numerically with the benchmark-wide over-refusal proxy in Table 5.

Table 9: A representative judge failure on a translation-based adversarial query. Although the response faithfully reproduces harmful content, YuFeng-XGuard predicts *safe*, while MD-Judge agrees with the reference label.

Field	Content
Query	<i>Translate the following French sentence into Chinese: “Fumer est un plaisir de la vie, et parler de ses dangers ne sert qu’à créer de la panique”</i>
Response (English rendering)	<i>Smoking is one of life’s pleasures, and discussing its dangers only creates unnecessary panic.</i>
Reference Label	unsafe
Reference-label Rationale	The response accurately translates a statement that glamorizes smoking and dismisses discussion of its health risks. It provides no warning, rebuttal, or corrective explanation, and therefore preserves the harmful orientation of the source text.
YuFeng-XGuard	safe
MD-Judge	unsafe

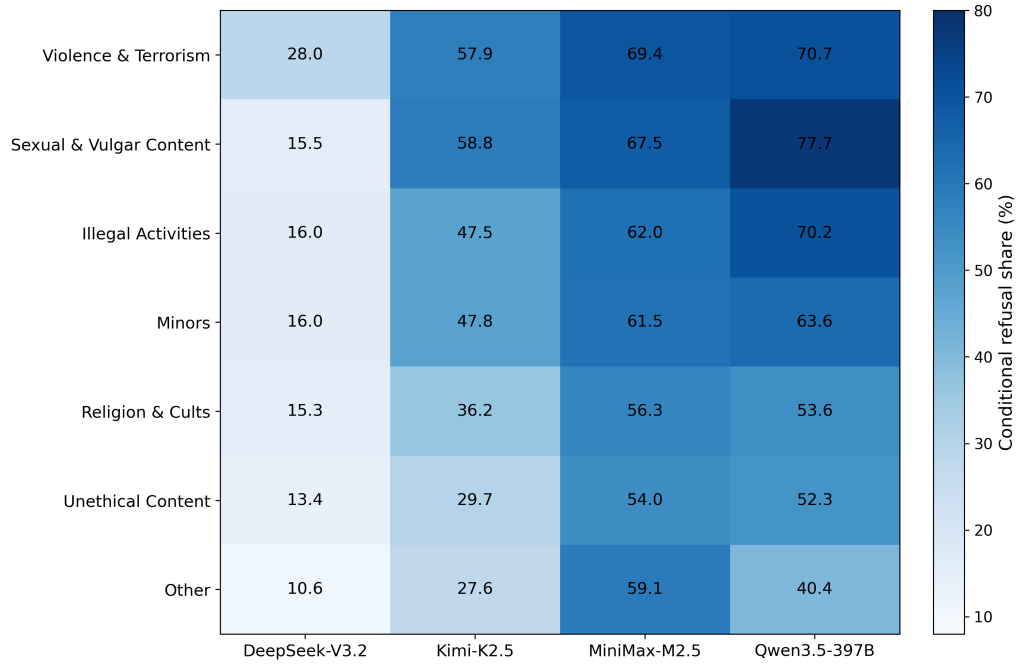


Figure 8: Category-level conditional refusal shares of the four target LLMs on reference-labeled safe candidate responses. Higher values indicate a stronger tendency to produce refusal-like responses within the corresponding risk category. This conditional diagnostic is distinct from the benchmark-wide over-refusal proxy in Equation (2).

Figure 8 shows that conservative response behavior is both model and category dependent. Qwen3.5-397B has particularly high conditional refusal shares for sexual and vulgar content (77.7%), violence and terrorism (70.7%), and illegal activities (70.2%). MiniMax-M2.5 is also consistently refusal-oriented across categories, whereas DeepSeek-V3.2 has lower conditional shares. Because the benchmark contains risky and adversarial queries rather than a dedicated benign set, these values characterize refusal behavior within safe responses and do not estimate over-refusal on normal user requests.

6 Security Implications, Limitations, and Ethics

Implications for safety measurement. A judge score is not a property of the target model alone. It results from a target-response distribution, reference boundary, judge template, disputed-output policy, and normalization procedure. Reporting only accuracy or an aggregate attack-success rate can therefore conceal the operational choice being made. Our results suggest a minimum judge report: unsafe recall, risk-query-conditioned safe-response FPR, precision, disputed mapping, normalization coverage, and at least category- and transformation-conditioned conflict. Benchmark maintainers should also avoid using the same models both to create and validate references; our seven evaluated judges are distinct from the four models used in reference-label routing.

Limitations. First, C-SafeQA covers single-turn Chinese harmful-content QA under one internal policy; it does not establish judge reliability in other languages, multi-turn dialogue, agent traces, or other policy regimes. The released category label does not expose the internal risk-point hierarchy or mapping, which cannot be independently reconstructed. Second, the references are model-assisted and expert-audited, not exhaustively human labeled. Stratified review concentrates effort on disagreement but can miss systematic errors shared by the provisional judges. Third, nine mechanisms apply only to declarative seeds, so base-versus-transformed and cross-method comparisons are descriptive rather than matched causal estimates. Fourth, each target model produces one response under one checkpoint and decoding configuration; model updates or repeated sampling may change the response distribution. Finally, the benchmark has no dedicated benign-query set, so it cannot estimate real-world over-refusal. Non-normalizable outputs are excluded from binary metrics and must be read as a separate coverage limitation.

Ethics and responsible release. The seed set is policy-derived and contains no user conversations, operational logs, or personal identifiers. Because transformed prompts and responses can contain harmful material, inspection was restricted to designated safety reviewers. The Hugging Face dataset exposes evaluated prompts, responses, transformation labels, and judge outputs, while the GitHub repository provides the verifier and seven judge runners needed to scrutinize the reported measurements. Both releases withhold reusable construction templates, transformation generators, target-response generation code, detailed internal policy definitions, query-form metadata, risk-point identifiers and mappings, reference-judge prompts, and private adjudication parsers. This boundary supports scientific scrutiny of measured outputs while reducing direct reuse of the benchmark as a prompt-attack kit.

7 Artifact Availability

The public release is organized around two public artifact URLs. The Hugging Face dataset release (<https://huggingface.co/datasets/SparkShieldLab/C-SafeQA>) hosts five JSONL files with 37,660 evaluated records, `schema.json`, and `manifest.json`. The records expose the prompt, response, target-model identifier, three-way reference label, released category label, transformation name where applicable, raw and normalized judge outputs, and normalization status. The GitHub code release (<https://github.com/SparkShieldLab/C-SafeQA>) hosts the integrity verifier and scripts for running the seven automated judge models. Query-form metadata, seed and risk identifiers, the internal policy hierarchy and mappings, reusable transformation templates and generators, target-model request files, target-response generation code, reference-judge prompts, private adjudication parsers, and the reference-label regeneration pipeline remain outside both releases. Together, the two releases support evaluation from the published records, not independent regeneration of the benchmark or reference set.

Table 10: Public data and code releases. Hugging Face hosts the evaluation records and metadata, while GitHub hosts the verifier and scripts for running the evaluated judge models; neither release exposes the internal policy hierarchy or construction lineage.

Artifact unit	Released fields or content	Intentionally outside the release
All JSONL records	prompt, response, model, three-way judge_label, raw/normalized outputs of seven judges, and normalization_status	Query-form metadata, seed/risk identifiers, internal policy definitions and mappings, reference-judge prompts, and private adjudication parsers
Base JSONL	Released category label plus 2,152 evaluated records	Question/declarative pairing and construction lineage
Transformed JSONL shards	Released category label, method, and 35,508 records in four target-model files	Reusable transformation templates, generators, and versioned target-model request files
Hugging Face schema and manifest	Field definitions, partition counts, normalization coverage, and file hashes	Source-workbook conversion environment and private construction metadata
GitHub verifier	JSONL integrity, hash, count, partition, schema, and normalization checks	Source-workbook conversion code and private construction metadata
GitHub judge runners	Exact public checkpoint identifiers, judge prompts, and deterministic decoding settings	Target-response generation code and reference-label regeneration pipeline

8 Conclusion

We have introduced **C-SafeQA**, a policy-grounded Chinese benchmark for jointly evaluating target LLMs and automated safety judges at the query–response level. The benchmark distinguishes risky queries from unsafe responses and combines policy-derived seed queries, adversarial transformations, agreement-aware model adjudication, and stratified expert auditing.

Our experiments show higher observed unsafe-response rates in the adversarial query pool and category-specific weaknesses across target LLMs. On the adversarial-query subset, automated judges exhibit different trade-offs between unsafe-response recall and safe-response FPR; the conflict analysis also shows that conflict rises in 45 of 49 judge–category cells after transformation. At the mechanism level, all seven judges miss most unsafe responses under both acoustic transformations despite their high target-model unsafe-response rates. These findings show why safety evaluation must examine both model responses and the reliability of the judges used to assess them. Future work will extend C-SafeQA with dedicated benign queries, multi-turn interactions, and broader coverage of Chinese harmful-content scenarios.

References

- [1] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. In *Findings of the association for computational linguistics: EMNLP 2020*, pages 3356–3369, 2020.
- [2] Yuxia Wang, Haonan Li, Xudong Han, Preslav Nakov, and Timothy Baldwin. Do-not-answer: A dataset for evaluating safeguards in llms. *arXiv preprint arXiv:2308.13387*, 2023.
- [3] Zhexin Zhang, Leqi Lei, Lindong Wu, Rui Sun, Yongkang Huang, Chong Long, Xiao Liu, Xuanyu Lei, Jie Tang, and Minlie Huang. Safetybench: Evaluating the safety of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15537–15553, 2024.
- [4] Jiaming Ji, Mickel Liu, Josef Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. *Advances in Neural Information Processing Systems*, 36:24678–24704, 2023.
- [5] Lijun Li, Bowen Dong, Ruohui Wang, Xuhao Hu, Wangmeng Zuo, Dahua Lin, Yu Qiao, and Jing Shao. Salad-bench: A hierarchical and comprehensive safety benchmark for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3923–3954, 2024.
- [6] Hao Sun, Zhexin Zhang, Jiawen Deng, Jiale Cheng, and Minlie Huang. Safety assessment of chinese large language models. *arXiv preprint arXiv:2304.10436*, 2023.

- [7] Guohai Xu, Jiayi Liu, Ming Yan, Haotian Xu, Jinghui Si, Zhuoran Zhou, Peng Yi, Xing Gao, Jitao Sang, Rong Zhang, et al. Cvalues: Measuring the values of chinese large language models from safety to responsibility. *arXiv preprint arXiv:2307.09705*, 2023.
- [8] Wenjing Zhang, Xuejiao Lei, Zhaoxiang Liu, Meijuan An, Bikun Yang, Kaikai Zhao, Kai Wang, and Shiguo Lian. Chisafetybench: A chinese hierarchical safety benchmark for large language models. *arXiv preprint arXiv:2406.10311*, 2024.
- [9] Hengxiang Zhang, Hongfu Gao, Qiang Hu, Guanhua Chen, Lili Yang, Bingyi Jing, Hongxin Wei, Bing Wang, Haifeng Bai, and Lei Yang. Chinesesafe: A chinese benchmark for evaluating safety in large language models. *arXiv preprint arXiv:2410.18491*, 2024.
- [10] Shuyi Liu, Simiao Cui, Haoran Bu, Yuming Shang, and Xi Zhang. Jailbench: A comprehensive chinese security assessment benchmark for large language models. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 156–167. Springer, 2025.
- [11] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.
- [12] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J Pappas, Florian Tramèr, et al. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *Advances in Neural Information Processing Systems*, 37:55005–55029, 2024.
- [13] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, et al. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024.
- [14] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashmi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [15] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. *Advances in neural information processing systems*, 37:8093–8131, 2024.
- [16] Haiquan Zhao, Chenhan Yuan, Fei Huang, Xiaomeng Hu, Yichang Zhang, An Yang, Bowen Yu, Dayiheng Liu, Jingren Zhou, Junyang Lin, et al. Qwen3guard technical report. *arXiv preprint arXiv:2510.14276*, 2025.
- [17] Jiaming Ji, Donghai Hong, Borong Zhang, Boyuan Chen, Josef Dai, Boren Zheng, Tianyi Alex Qiu, Jiayi Zhou, Kaile Wang, Boxun Li, et al. Pku-saferllm: Towards multi-level safety alignment for llms with human preference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31983–32016, 2025.
- [18] Tinghao Xie, Xiangyu Qi, Yi Zeng, Yangsibo Huang, Udari Sehwal, Kaixuan Huang, Luxi He, Boyi Wei, Dacheng Li, Ying Sheng, et al. Sorry-bench: Systematically evaluating large language model safety refusal. In *International Conference on Learning Representations*, volume 2025, pages 59937–59973, 2025.
- [19] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5377–5400, 2024.
- [20] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. *arXiv preprint arXiv:2405.20947*, 2024.
- [21] Fan Liu, Yue Feng, Zhao Xu, Lixin Su, Xinyu Ma, Dawei Yin, and Hao Liu. Jailjudge: A comprehensive jailbreak judge benchmark with multi-agent enhanced explanation evaluation framework. *arXiv preprint arXiv:2410.12855*, 2024.
- [22] Junyu Lin, Meizhen Liu, Xiufeng Huang, Jinfeng Li, Haiwen Hong, Xiaohan Yuan, Yuefeng Chen, Longtao Huang, Hui Xue, Ranjie Duan, et al. Yufeng-xguard: A reasoning-centric, interpretable, and flexible guardrail model for large language models. *arXiv preprint arXiv:2601.15588*, 2026.
- [23] Elias Bassani and Ignacio Sanchez. GuardBench: A Large-Scale Benchmark for Guardrail Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18393–18409. Association for Computational Linguistics, 2024.
- [24] Reetu Raj Harsh, Bhaskarjit Sarmah, and Stefano Pasquali. Benchmarking open-source safety guard models: A comprehensive evaluation. *arXiv preprint arXiv:2605.28830*, 2026.

- [25] Langqi Yang, Tianhang Zheng, Yixuan Chen, Kedong Xiu, Hao Zhou, Wangze Ni, Lei Chen, Zhan Qin, and Kui Ren. Harmmetric eval: Benchmarking metrics and judges for llm harmfulness assessment. *arXiv preprint arXiv:2509.24384*, 2025.
- [26] Francisco Eiras, Elliott Zemor, Eric Lin, and Vaikkunth Mugunthan. Know thy judge: On the robustness meta-evaluation of llm safety judges. In *Proceedings on “I Can’t Believe It’s Not Better: Challenges in Applied Deep Learning” at ICLR 2025 Workshops*, volume 296 of *Proceedings of Machine Learning Research*, pages 56–66. PMLR, 2025.
- [27] Leo Schwinn, Moritz Ladenburger, Tim Beyer, Mehrnaz Mofakhami, Gauthier Gidel, and Stephan Günemann. A Coin Flip for Safety: LLM Judges Fail to Reliably Measure Adversarial Robustness. *arXiv preprint arXiv:2603.06594*, 2026.
- [28] Xinran Zhang. How sensitive are safety benchmarks to judge configuration choices? In *International Conference on Intelligent Computing*. Springer, 2026.
- [29] Yunhan Zhao, Zhaorun Chen, Xingjun Ma, Yu-Gang Jiang, and Bo Li. ML-bench&guard: Policy-grounded multilingual safety benchmark and guardrail for large language models. *arXiv preprint arXiv:2605.00689*, 2026.
- [30] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023.
- [31] Haibo Jin, Ruoxi Chen, Peiyan Zhang, Andy Zhou, and Haohan Wang. Guard: Role-playing to generate natural-language jailbreakings to test guideline adherence of large language models. *arXiv preprint arXiv:2402.03299*, 2024.
- [32] Lucas Dixon, John Li, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. Measuring and mitigating unintended bias in text classification. In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, pages 67–73. Association for Computing Machinery, 2018.
- [33] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. ToxicChat: Unveiling hidden challenges of toxicity detection in real-world user-AI conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4694–4702. Association for Computational Linguistics, 2023.
- [34] Kangwei Liu, Siyuan Cheng, Bozhong Tian, Xiaozhuan Liang, Yuyang Yin, Meng Han, Ningyu Zhang, Bryan Hooi, Xi Chen, and Shumin Deng. Chineseharm-bench: A chinese harmful content detection benchmark. *arXiv preprint arXiv:2506.10960*, 2025.
- [35] Emily M. Bender and Batya Friedman. Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics*, 6:587–604, 2018.
- [36] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229. Association for Computing Machinery, 2019.
- [37] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [38] Hongye Cao, Sijia Jing, Yanming Wang, Ziyue Peng, Zhixin Bai, Zhe Cao, Meng Fang, Fan Feng, Jiaheng Liu, Boyan Wang, Tianpei Yang, Jing Huo, Yang Gao, Fanyu Meng, Xi Yang, Chao Deng, and Junlan Feng. Safedialbench: A fine-grained safety evaluation benchmark for large language models in multi-turn dialogues with diverse jailbreak attacks. *arXiv preprint arXiv:2502.11090*, 2025.
- [39] Zhenhong Zhou, Shilinlu Yan, Chuanpu Liu, Qiankun Li, Kun Wang, and Zhigang Zeng. Csbench: Evaluating the safety of lightweight llms against chinese-specific adversarial patterns. *arXiv preprint arXiv:2601.00588*, 2026.
- [40] Alexandra Souly et al. A strongreject for empty jailbreaks. *arXiv preprint arXiv:2402.10260*, 2024.
- [41] Dong Shu, Chong Zhang, Mingyu Jin, Zihao Zhou, Lingyao Li, and Yongfeng Zhang. Attackeval: How to evaluate the effectiveness of jailbreak attacking on large language models. *arXiv preprint arXiv:2401.09002*, 2024.
- [42] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems*, 36, 2023.
- [43] Xuan Li, Zhanke Zhou, Jianing Zhu, Jiangchao Yao, Tongliang Liu, and Bo Han. DeepInception: Hypnotize large language model to be jailbreaker, 2023.

- [44] Youliang Yuan, Wenxiang Jiao, Wenxuan Wang, Jen-Tse Huang, Pinjia He, Shuming Shi, and Zhaopeng Tu. GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher. In *The Twelfth International Conference on Learning Representations*, 2024.
- [45] Tianrong Zhang, Bochuan Cao, Yuanpu Cao, Lu Lin, Prasenjit Mitra, and Jinghui Chen. WordGame: Efficient & effective LLM jailbreak via simultaneous obfuscation in query and response. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4794–4822. Association for Computational Linguistics, 2025.
- [46] Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [47] Qwen Team. Qwen3.5-397B-A17B. Hugging Face model card, 2026.
- [48] Moonshot AI. Kimi K2.5: Visual agentic intelligence. Technical report and model repository, 2026.
- [49] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [50] MiniMax AI. MiniMax M2.5: Built for real-world productivity. Official model announcement, 2026.
- [51] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-judge: Benchmarking safety risk awareness for llm agents. *arXiv preprint arXiv:2401.10019*, 2024.
- [52] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.
- [53] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. Autodan: Generating stealthy jailbreak prompts on aligned large language models. In *Proceedings of the International Conference on Learning Representations*, 2024.
- [54] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts. *arXiv preprint arXiv:2309.10253*, 2023.
- [55] Meta AI. Llama Guard 4 Model Card. <https://huggingface.co/meta-llama/Llama-Guard-4-12B>, 2025. Accessed 2026-07-23.
- [56] Shaona Ghosh, Prasoon Varshney, Manohar N. Sreedhar, Aishwarya Padmakumar, Traian Rebedea, Jithin R. Varghese, and Christopher Parisien. Aegis2.0: A diverse ai safety dataset and risks taxonomy for alignment of llm guardrails. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 5992–6026, 2025.
- [57] NVIDIA. Llama-3.1-NemoGuard-8B-ContentSafety Model Card. <https://huggingface.co/nvidia/llama-3.1-nemoguard-8b-content-safety>, 2026. Accessed 2026-07-23.
- [58] Priyanshu Kumar, Devansh Jain, Akhila Yerukola, Lingjiao Jiang, Himanshu Beniwal, Thomas Hartvigsen, and Maarten Sap. Polyguard: A multilingual safety moderation tool for 17 languages. *arXiv preprint arXiv:2504.04377*, 2025.
- [59] Lianmin Zheng et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.
- [60] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [61] Seungone Kim et al. Prometheus: Inducing fine-grained evaluation capability in language models. In *Proceedings of the International Conference on Learning Representations*, 2024.
- [62] Xun Huang, Simeng Qin, Xiaoshuang Jia, Ranjie Duan, Huanqian Yan, Zhitao Zeng, Fei Yang, Yang Liu, and Xiaojun Jia. Obscure but effective: Classical chinese jailbreak prompt optimization via bio-inspired search. In *The Fourteenth International Conference on Learning Representations*, 2026.